Cockpit System Audit & Orchestration Advice

Cockpit System Audit & Orchestration Advice

Date: 2026-05-17 · **Repo:** claude-cockpit @ `develop` (`9129743`) · **Audience:** project owner

This report answers one question: *which cockpit features actually work, why the ones that don't never did, and what to do to reach the stated goal — a reliable multi-provider orchestrator where one captain (any provider) controls many crew (any provider), that auto-recovers and auto-learns.*

It combines a state-level empirical sweep, two deep code audits (multi-provider spawn path; captain↔crew control loop), and the prior idle-detection research report.

Executive Summary

The codebase is **green** (318/320 tests pass; the 2 failures are a known stale assertion, #70/#76). Synchronous CLI commands work well. But **every autonomous, periodic, or cross-agent behavior is either broken or never ran**, and they all share **one root cause**:

> Cockpit has no always-running, cockpit-owned process. Every reliability-critical behavior depends on an agent voluntarily remembering to act, observed via terminal scraping. When no agent is babysitting a `/loop`, everything silently freezes — with no detection and no recovery.

The four goals — reliable orchestration, multi-provider, auto-recovery, auto-learn — are **not four problems. They are one missing architectural layer.**

Method

Angle	What was done
Empirical state sweep	Ran every safe CLI command; inspected runtime state files, mtimes, and run history under `~/.config/cockpit` and `~/cockpit-hub`
Code audit A	Deep audit of the multi-provider spawn path (`src/drivers` , `src/runtimes` , `src/commands/launch.ts` , `crew.ts` , `config.ts` , `src/projection`)
Code audit B	Deep audit of the captain↔crew control loop (dispatch → work → completion → collection), `src/reactor` , classifier, handoff scripts
Prior research	`docs/research/2026-05-16-idle-detection-and-inter-agent-orchestration.md` (notchi/cmux/A2A patterns)
Test suite	Full `vitest run`

Part 1 — Feature Inventory

Working (verified by execution)

Feature	Status	Evidence
`doctor`	Works	Runs 39 checks; the FAILs it reports are *accurate*
`init`	Works	Runtime self-heal verified (PR #74)
`projects` / `status`	Works	Render correctly
`standup`	Works	Real git data, zero-token (9 real commits when scoped to cockpit)
`retro`	Works well	Aggregated 30 commits / 9 merged PRs from a real project
`dashboard --once/--pane`	Works	Renders grid (data stale — see Part 1.3)
`projection` (list/diff/emit)	Works	Targets writable, diffs render
`tracker` (gh issues/PRs/checks)	Works	gh authenticated
`workspace` / `runtime` bridges	Works	`runtime` correctly reports command session *stopped*
Test suite	Green	318/320 pass; 2 fails = stale config test (#70/#76)

Never worked / never used (zero history on disk)

Feature	Evidence
Reactor engine	`reactor status` → "Last poll: **never run**", "Repos watched: **0**". One 3-byte empty event file from May 5. Never configured.
Learnings / self-improvement	`~/cockpit-hub/learnings/` **empty since Mar 31**. Zero learnings ever recorded despite 4 scripts + `command learnings-review`.
Wiki knowledge system	`~/cockpit-hub/wiki/pages/` **empty since Apr 7**. Zero pages ingested despite 3 scripts + `command wiki-aggregate`.

Worked once, then decayed (live-session dependent)

Feature	Evidence
Auto-status (`poll-status`)	Single batch write **May 15 22:31**, frozen since. All 12 spokes stale in `doctor`.
Dashboard data	Renders, but every project shows `offline/stale` — reading 2-day-old `status.md`.
Daily briefing	Last `daily-logs/` entry **May 5** — 12 days dark.
Notifications	Code is correct; cmux healthy (`PONG`). But the command workspace is **stopped**, so notifications have no destination. The `doctor` "Notifier FAIL" is a *correct report*, not a bug.

> **The pattern:** every feature in the bottom two tables assumes "an agent will be alive and will remember to drive me." None owns its own execution. Adding more features on this substrate reproduces the same outcome.

Part 2 — Audit A: Captain → Crew Control Loop

The loop today: **dispatch works; completion/collection does not exist.**

DISPATCH (works)	WORK (opaque)	COMPLETION (broken)
cockpit crew spawn → cmux new tab → type cmd + Enter → sleep 3000ms (!) → type task + Enter	independent Claude CLI in a cmux tab no PID, no exit code, no env markers, no transcript handle	regex-scrape the *captain's* pane (not the crew's!) no "done" pattern exists at all

What works

`cockpit crew spawn` creates a cmux surface, renames the tab `🔧 <project>:<name>`, and does a two-step terminal send (launch command, **hardcoded 3000 ms sleep**, then task text). Crew identity lives **only in the cmux tab title** (`crew.ts:24–34`); the rename is best-effort and swallows errors (`cmux.ts:107–109`) — if it fails, the crew is unfindable forever.

What does not work

- **Completion detection = one heuristic.** `auto-status.ts:73` reads `driver.readScreen(captainWorkspace)` — the **captain's** pane, never the crew's. `status-classifier.ts` regex-classifies the last 50 lines into `offline/errored/blocked/busy/idle`. **There is no "done" / "task complete" pattern — `idle` is just the no-match fallback (`status-classifier.ts:92`).** A finished crew is indistinguishable from an idle prompt.
- **No reverse channel.** `crew_signals` frontmatter: grep returns nothing — it does not exist in code. Handoff scripts are captain day-to-day continuity, unrelated to crew. The only crew→captain path is the crew agent *choosing* to type `cockpit runtime send` — in no enforced template.
- **No scheduler.** `grep cron|launchd|setInterval` → nothing. The "always-on" reactor loop is a *manual instruction to an agent* in `reactor-ops/SKILL.md:155` to use `/loop`, not infrastructure.
- **No timeout / watchdog / heartbeat / retry** anywhere (grep-confirmed absent).

Failure modes (concrete)

Scenario	What actually happens
Crew hangs	No timeout. Tab sits forever. Classifier shows `busy` (spinner) or `idle`. Nothing escalates.
Crew crashes	Only detectable if reactor polled the *crew* pane — it polls the *captain* pane, so never. Dead tab persists.
Captain not looking	Result lost until captain manually runs `crew read`. No push, no notification.
cmux unresponsive	`execSync` has no timeout → a hung call blocks the whole `cockpit` process indefinitely; reads silently misclassify as `offline`.
Crew finishes during captain compaction	Completion never recorded anywhere; no sentinel persists it. Status drifts permanently (this is issue #19).

> **#64 (the reliability fix) is fully designed but the implementation was scrapped on 2026-05-16** (PRs #71/#69/#67 abandoned). The design + 1032-line plan exist as docs only. **None of it is in the codebase.** You have been here before — this matters for the recommendation.

Part 3 — Audit B: Multi-Provider Reality

The abstraction surface (driver / runtime / projection interfaces, capability registry) is **clean and extensible**. Functionally, cockpit is **Claude-only**.

Provider	Driver	Probe	Captain spawn	Crew spawn	Verdict
claude	Yes	Real	Wired, special-cased	Wired, interactive	**End-to-end**
opencode	Yes	Real	Absent (generic branch)	Interactive, but global identity leak (#61)	**Partial (crew only)**
codex	Yes	Real	Broken	Print-mode, dies after 1 turn	**Stub-grade**
gemini	Yes	Real	Broken	Print-mode, dies after 1 turn	**Stub-grade**
aider	Yes	Real	Broken	Print-mode, no role file	**Stub-grade**
cursor	No driver	—	Impossible	Impossible (`crew.ts` throws)	**Projection-only**

The hardcoded-to-Claude assumptions

- `launch.ts:128` hard-branches `if (driver.name === "claude")`. Only Claude gets: persistent interactive session, role via `--append-system-prompt-file`, `--plugin-dir` skills, and `claude -c` resume. Everything else falls through to a **one-shot print-mode command** with the role as a prompt string — it dies after one turn and cannot orchestrate or be orchestrated.
- `scripts/spawn-workspace.sh` `case "$AGENT"` : **opencode is not handled →** `exit 1`. Startup checklist injection is gated `if [ "$AGENT" = "claude" ]`.
- `crew.ts:139` : `interactive = agent.name === "claude" || agent.name === "opencode"`. codex/gemini/aider crews are non-interactive and unmanageable.

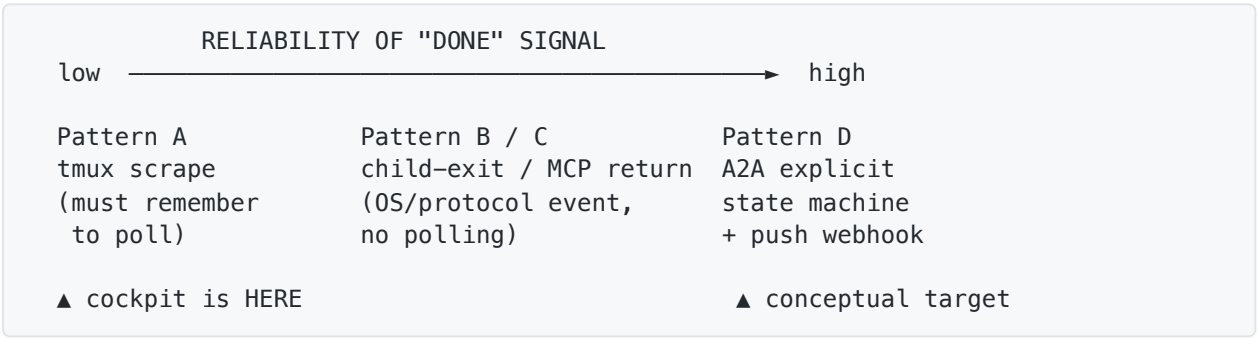
- **Capability gating is dead code.** `validateRole()` / `probeAll()` exist (`registry.ts:40-67`) but **no spawn path ever calls them**. A captain is spawned with whatever the config names, zero validation.
- Plugin/skills (`cockpit:captain-ops`, etc.) are Claude-Skill-tool namespaced. A non-Claude captain has a 45-line generic file and **none of the operational playbook**.
- Config schema *can* express `captain: {agent: X}`, but **crew provider is CLI-flag only** (`--agent`), never config-driven per project/role.

> **Heterogeneous captain(X) → crew(Y) is not expressible today**, capability gating is unenforced, and a non-Claude captain has no working orchestration runtime. Issues #35 and #61 (both OPEN) are the blocking tracked work. opencode is the only viable non-Claude provider, and even it leaks role identity globally.

Part 4 — Research Synthesis (Idle Detection & Orchestration)

From the prior research report (notchi, cmux, A2A, opencode):

> **Both notchi and cmux — the two tools cockpit's environment is built around — independently converged on the same answer: do not scrape terminals. Install agent-native lifecycle hooks that emit a structured event over a local socket; normalize all agents into one event vocabulary.**



Key levers identified by the research:

- **Hook-based completion** (notchi/cmux model): merged, idempotent `Stop` / `SubagentStop` / `SessionEnd` hooks POST a normalized JSON envelope to a local socket. Zero polling for Claude.
- **Invocation-based** (Pattern B/C): run delegated work headless — `claude -p --output-format json` or **`opencode serve`** + synchronous **`POST /session/:id/message`** (blocks until done, returns result). Completion = process exit / HTTP return, **identical for any provider**.
- **opencode `serve` is the single biggest lever** — synchronous wait endpoint + SSE + status polling + arbitrary cheap/local model. Reliability *and* cost *and* heterogeneity in one move.
- **Semantic trap (cmux #2576):** `Stop` ≠ "done" ≠ "needs input." Conflating them rebuilds cmux's most-hated bug. Must model *turn-complete* / *blocked-on-human* / *terminated* distinctly.

Part 5 — The Core Finding

Your four goals map to **one** missing layer:

Goal	Why it fails today	What fixes it
Reliable orchestration	Scraping the wrong pane, no "done" signal, no timeout	Crew emits hook event → socket → task state; captain *queries state*, never scrapes; hard timeout
Multi-provider	Spawn path special-cases Claude; others die after 1 turn	Headless invocation (`claude -p` / `opencode serve`) — completion is provider-agnostic by construction
Auto-recovery	No daemon, no watchdog, no health layer	A rule on the same event bus: `task.failed` / `timeout` → reassign/restart (this *is* #77)
Auto-learn	wiki/learnings depend on an agent remembering	Subscribe wiki/learnings to bus events (`task.done` → capture); curate with a prompt, not silently

> **They are not four features. They are one substrate.** Build it once and three of the four fall out almost for free. Build none of it and every future feature joins the "never worked" list.

Part 6 — Recommended Direction

The reframe: a nervous system, not a brain

Your principle — *"the CLI is the ignition key, not the brain; orchestration happens in the agent session"* — conflicts with reliability, because an agent session is mortal and forgetful. The resolution is **not** to move the brain. It is to add a thin, dumb, always-running **control plane** that does not think:

- transports events over a local socket,
- persists task state to disk,
- enforces timeouts and restarts dead things.

The agent stays the brain. The daemon is the **nervous system** that makes the brain's intent survive its own death. This is exactly notchi's/cmux's converged design.

Sequenced plan

1 — Build the substrate as ONE thin vertical slice (not the whole #64 plan). The prior attempt failed because it was a large plan bolted onto the scraping architecture. Do the opposite: smallest end-to-end slice that proves the model.

- A cockpit-owned long-lived process (launchd-managed — survives session death; **this piece has never existed**).
- One Unix socket, one event vocabulary: `task.started / task.done / task.failed / task.blocked`.
- Claude crew gets a merged, idempotent `Stop` / `SubagentStop` hook (notchi's exact pattern) that POSTs to the socket.
- A JSON task-state file the captain *reads* instead of scraping.
- One hard timeout → `task.failed` if no event in N minutes.

That slice alone fixes orchestration pain points #1/#2/#3 for Claude→Claude and gives the bus everything else hangs off.

2 — Second provider = opencode via `serve`, not interactive tabs. opencode is the only non-Claude with a true synchronous "wait until done" endpoint. Heterogeneous that actually works = Claude captain blocking on `opencode serve` workers. Defer codex/gemini/aider (print-mode-only; not viable as managed crew without #35).

3 — Re-point reactor / status / wiki / learnings at the bus. They don't need fixing — they need a *trigger*. Once events flow, `poll-status` becomes event-driven, the dashboard unfreezes, learnings auto-capture on `task.done`. **Decide remove-vs-keep here:** if wiki/learnings aren't worth subscribing to the bus, delete them — they have never been used.

4 — Wire capability gating into the spawn path. `validateRole` already exists and is never called. One line of enforcement turns “spawn whatever and hope” into “refuse a codex captain because it can’t auto-approve.” Cheap, high value.

Defer / push back on: full A2A protocol (right long-term target, wrong now — the JSON state store is the migration path). Do not rebuild the elaborate #64 plan in one shot. **Add no new feature surface until the substrate carries the existing ones.**

Part 7 — Risks & Honest Pushback

Risk	Detail
Daemon is new territory	The launchd-managed process is the one load-bearing piece that has never existed. That is the real work; everything else is mechanical once it exists.
Semantic trap	If `Stop` / blocked-on-human / terminated are not modelled distinctly from day one, you rebuild cmux issue #2576 (every parked session screams "needs input").
Loss of observability	Headless delegation (Pattern B/C) trades live cmux-tab watching for reliability. opencode SSE mitigates. Decide consciously: recommendation is reliable-by-default with an opt-in observable mode.
Repeating history	The #64 attempt was scrapped once. The mitigation is *scope*: one vertical slice that replaces the channel, not a 1000-line plan layered on scraping.

Appendix — Issue Cross-Reference

Issue	State	Relevance
#64	OPEN (bug)	Crew→captain completion signal. Fully designed, **unimplemented**, prior attempt scrapped.
#77	OPEN (P1)	Service health tracking — becomes trivial *once the bus exists*.
#35	OPEN (P3)	Captain/crew role identity for non-Claude — core multi-provider blocker.
#61	OPEN	opencode session-scoped identity — global leak today.
#31	CLOSED	Projection V1 — the only delivered multi-agent piece; emits static text only.
#19	OPEN	Captain post-compaction status drift — a symptom of no event persistence.
#18	OPEN (bug)	command→captain send-submit asymmetry — same fragile terminal path.
#70 / #76	OPEN	Stale config test (the only 2 test failures; not a real defect).

Appendix — Key Evidence Locations

- Completion classifier (no “done” pattern): `src/reactor/status-classifier.ts:64-93`
- Reads captain pane not crew: `src/reactor/auto-status.ts:73`
- Claude hard-branch: `src/commands/launch.ts:128`
- Crew interactive gate: `src/commands/crew.ts:139`
- Capability gating dead code: `src/capabilities/registry.ts:40-67` (never called)
- opencode unhandled in shell spawn: `scripts/spawn-workspace.sh` (`*) → exit 1`)
- No scheduler: `grep cron|launchd|setInterval src/ plugin/` → empty
- Prior research: `docs/research/2026-05-16-idle-detection-and-inter-agent-orchestration.md`